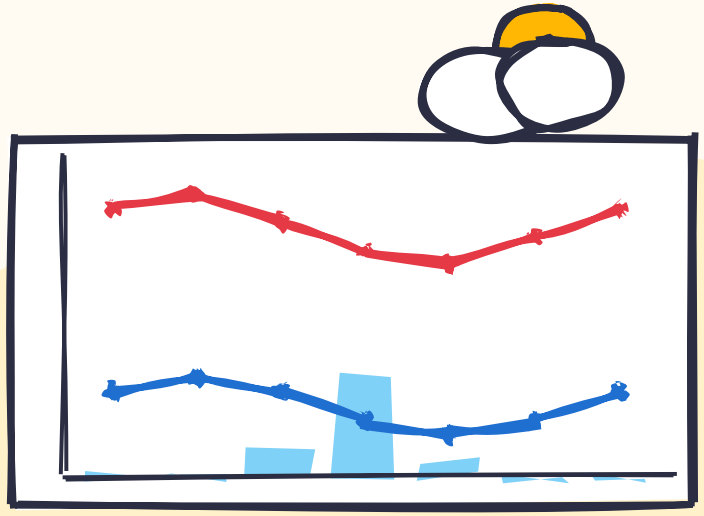
Young Innovators · Class 30

Python Weather API Dashboard

Ask a real weather server. Plot the answer.

APIs JSON Dashboards



Did you know?
Weather apps don't measure the weather themselves. They ask weather services through APIs, millions of times a second.

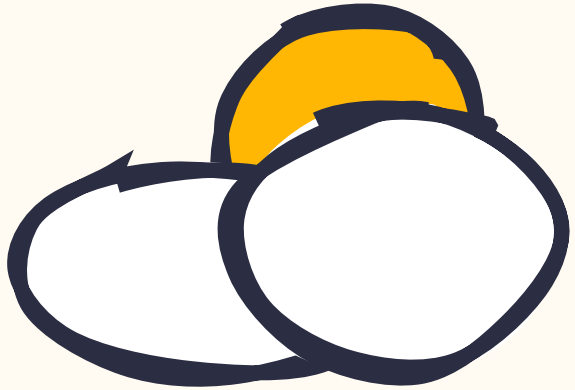
LEARN

Let's think...

Where do weather apps get data?

Sports day decision

The principal wants the **driest, coolest day** next week for sports day. You could look outside... or ask a professional weather model for 7 days of forecasts in 1 second. How does a program 'ask' another computer?



Google it?

Guess!

Use an API!

Today: you write Python that calls a **real weather API**, reads the JSON reply, and plots a **7-day dashboard** for Pakistani cities.

Learning objectives

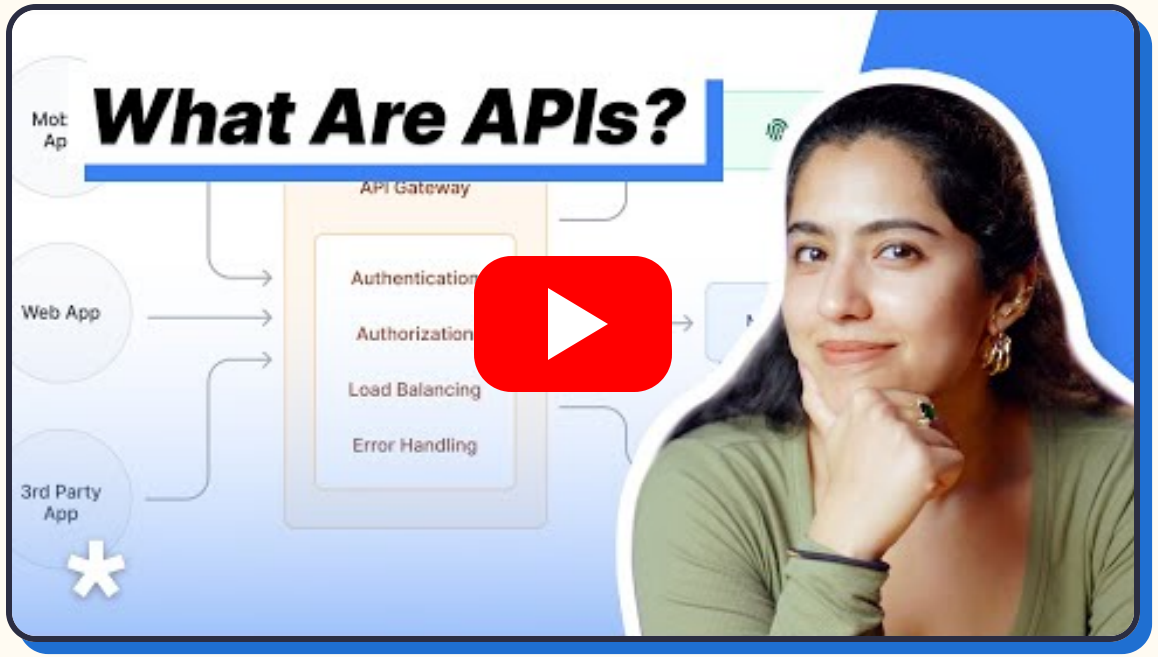
By the end of this class I can...

1	**What an API is:** explain an **API** as a request → response, with a **URL**, **parameters** and **JSON**	**Proof:** I can point to the parameters in a weather URL
2	**Fetch real data:** use **requests** to get a 7-day forecast and pull values out of **JSON**	**Proof:** my program prints tomorrow's max temperature for our city
3	**Dashboard:** plot a **weather dashboard** for several cities and make a decision from it	**Proof:** my chart shows the best day for our sports event

LEARN

Objective 1 · What an API is · video

Video time!



APIs Explained (in 4 Minutes)

Aced · about 4 minutes · youtu.be/bxuYDT-BWaI

Watch for...

1. What does an API **do**?
2. What is a **request** and a **response**?
3. Name one example of an API from the video.

Scan to watch

After the video: Think of an app you use. What data might it be asking another server for?

LEARN

Objective 1 · What an API is

An API is a menu for programs

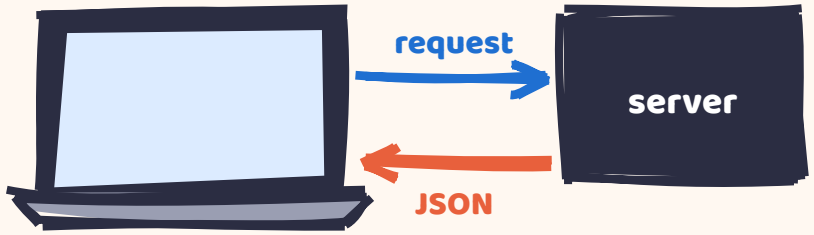
The idea

An **API** (Application Programming Interface) lets one program ask another for data. You send a **request** to a **URL**; the server sends back a **response**.

api.open-meteo.com/v1/forecast?
latitude=33.68&longitude=73.05&daily=temperature_2m_max

Everything after **?** are **parameters**, joined by **&**: where, and what you want. Open-Meteo is free with no account.

Example: ordering from a menu



In a restaurant you don't walk into the kitchen. You order from a menu, the waiter takes it, and food comes back. The API is the menu and the waiter.

LEARN

Objective 2 · Fetch real data

JSON: data in boxes inside boxes

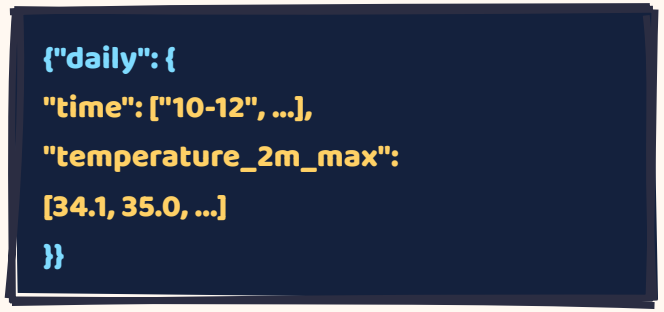
The idea

The reply is **JSON**: text that looks like Python **dictionaries** and **lists**:

```
{"daily": {"time": ["2026-10-12", ...],
  "temperature_2m_max": [34.1, 35.0, ...]}}
```

data = r.json() turns it into Python. Then dig in with keys and indexes: **data["daily"]["temperature_2m_max"][0]** → today's max.

Example: a school bag



Your bag (dictionary) has a pencil case (another dictionary) with pens in a row (a list). 'Bag → pencil case → 3rd pen' is how you dig into JSON.

LEARN

Objective 3 · Dashboard

A dashboard tells you what to do

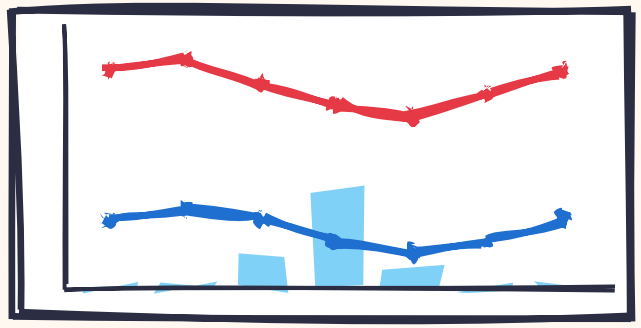
The idea

A **dashboard** puts the important numbers on one screen so someone can **decide**. For sports day:
Red line: max temperature. **Blue line**: min. **Bars**: rain (mm).

Good dashboards answer a question at a glance: label units, mark limits (e.g. 35 °C too hot), and keep it simple.

Remember: a forecast is a **prediction**, less certain further ahead.

Example: a car dashboard



A driver doesn't read the engine's computer logs. They glance at speed, fuel and warning lights, then decide. Good data design means decisions in seconds.

Real world

APIs power everything



Weather apps
Call forecast APIs every time you open them



Ride apps
Maps APIs calculate routes and prices



PMD
Pakistan Meteorological Department shares forecasts and warnings



Farmers' SMS alerts
Rain forecasts sent to farmers' phones



Which apps on your family's phone probably use a weather or maps API?

Free and open

Open-Meteo uses data from national weather services. Many governments share data openly so people can build tools.

LEARN

Quick quiz

API detective

Question	Answer
1. In a URL, what comes after the '?'	
2. What separates two parameters?	
3. What does r.json() give you?	
4. data['daily']['time'][0] is...?	
5. Is a 7-day forecast certain?	

Answers

Cover! 1: parameters · 2: & · 3: Python dicts and lists · 4: the first date · 5: no, less certain further ahead

Part 2 · Make it

Now let's put it to work!

You know the ideas. Time to play, build and test. Use these slides while you work:

1. Human API
2. Our setup
3. Ask the API, read the JSON
4. Plot the dashboard and decide
5. Compare cities and write a memo
6. Dashboard check
7. Data engineer challenges

MAKE

Class game

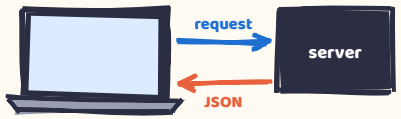
Human API

How to play

1. One student is the **weather server** with a printed forecast table for 3 cities.
2. Others are **apps**. They must send requests on slips in the exact format: CITY=Lahore&DAY=2&WANT=max
3. Server answers only correct requests, as JSON on a slip: {"max": 33}
4. Wrong format? Server replies {"error": "bad request"}

Then discuss

Why must requests follow an exact format? What did an 'error' reply teach you?



Big idea

APIs are strict so computers never misunderstand each other.

MAKE

Build it

Our setup

Thonny
Python 3

requests + matplotlib
Tools → Manage packages

Internet
To reach api.open-meteo.com

City coordinates
Islamabad 33.68, 73.05 ·
Lahore 31.55, 74.34 ·
Karachi 24.86, 67.01

Decision sheet
For the sports day memo

Browser
Paste the URL to see raw JSON

Try the URL in a browser first: you'll see the raw JSON your program receives.

Be polite
Don't call an API in a fast loop. One request per city per run is plenty; free APIs have limits.

MAKE

Objective 2 · Fetch real data · Build step 1 of 3

Ask the API, read the JSON

```
import requests

URL = "https://api.open-meteo.com/v1/forecast"
params = {
    "latitude": 33.68, "longitude": 73.05,       # Islamabad
    "daily": "temperature_2m_max,temperature_2m_min,precipitation_sum",
    "timezone": "Asia/Karachi",
}

r = requests.get(URL, params=params, timeout=10)
print(r.status_code)                # 200 = OK
data = r.json()

days = data["daily"]["time"]
tmax = data["daily"]["temperature_2m_max"]
tmin = data["daily"]["temperature_2m_min"]
rain = data["daily"]["precipitation_sum"]

for d, hi, lo, mm in zip(days, tmax, tmin, rain):
    print(f"{d}: {lo}–{hi} °C, rain {mm} mm")
```

params

requests builds the ?latitude=...&longitude=... part for you.

status

200 = OK. 400 = bad request (check spelling). No internet? ConnectionError.

zip()

Walks through the 4 lists together, one day at a time.

MAKE

Objective 3 · Dashboard · Build step 2 of 3

Plot the dashboard and decide

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(10, 5))
ax.bar(days, rain, color="skyblue", label="Rain (mm)")
ax2 = ax.twinx()
ax2.plot(days, tmax, "o-", color="red",  label="Max °C")
ax2.plot(days, tmin, "o-", color="blue", label="Min °C")
ax2.axhline(35, linestyle="--", color="orange")   # too hot
ax.set_ylabel("Rain (mm)");  ax2.set_ylabel("Temperature (°C)")
ax.set_title("Islamabad: next 7 days")
fig.legend(loc="upper left")
plt.xticks(rotation=45); plt.tight_layout()
plt.savefig("dashboard.png", dpi=150)
plt.show()

best = min(range(len(days)), key=lambda i: (rain[i], tmax[i]))
print("Best day for sports:", days[best])
```

Two axes

Rain bars on the left axis, temperatures on the right.

best

Picks the day with the least rain, then the lowest max temperature.

Your rule

Change the key to match YOUR definition of a good day.

MAKE

Objective 3 · Build step 3 of 3

Compare cities and write a memo

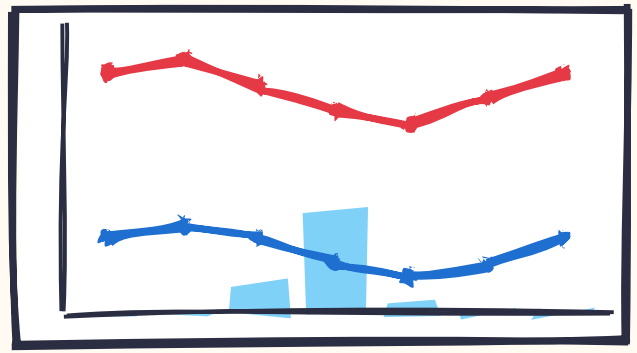
a Put the request in a function: **get_forecast(lat, lon)**.

b Loop over Islamabad, Lahore and Karachi. Plot all three max temperatures on one chart.

c Which city is hottest this week? Which has rain?

d Write a 3-line memo to the principal: best day, the evidence, one risk.

e Bonus: add today's date and 'Source: Open-Meteo' to your chart.



Check: Your memo names a day and backs it with forecast numbers.

MAKE

Test like an engineer

Dashboard check

Check	Done?	Notes
status_code is 200		
7 days printed correctly		
Chart: title, labels, units		
3 cities compared		
Memo with evidence		

Debug checklist

KeyError 'daily'? Print data: the API may be reporting an error message.
ModuleNotFoundError? Install requests / matplotlib.
Times look wrong? Add timezone=Asia/Karachi.

Always print the raw JSON once. Seeing the real structure saves hours of guessing.

MAKE

Level up!

Data engineer challenges

Choose your challenge



Heatwave alert: print a warning if any day is over 40 °C.



Air quality: Open-Meteo also has an air-quality API: plot PM2.5 for Lahore.



GUI: put the dashboard in a tkinter window with a city drop-down.



Compare: log your DHT22 readings next to the forecast. How accurate was it?

Think like an engineer

Why is the forecast for 7 days ahead less reliable than for tomorrow?

Boss level

Build an 'umbrella bot': runs each morning and tells you whether to take an umbrella.

Objective check

Can I really do it? Prove it.

01	Explain request, response, parameters and JSON using your URL.	○ ○ ○
02	Show your program printing the 7-day forecast. How did you get tomorrow's max?	○ ○ ○
03	Show your dashboard and read your memo to the principal.	○ ○ ○

Rate yourself

Colour one circle each. Red: not yet. Amber: with help. Green: I can explain it to a friend.

Teacher: Weather briefing: each group presents its sports-day recommendation in 60 seconds.

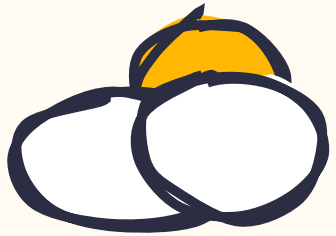
Show what you know

What did we learn?

- [x] An API lets programs ask other programs for data
- [x] URL + parameters = the question; JSON = the answer
- [x] r.json() → dicts and lists you can index
- [x] Dashboards help people decide quickly
- [x] Forecasts are predictions: say how sure you are

At home

Compare your weather app's forecast with yours tomorrow. Did they agree?



Exit ticket: in data['daily']['temperature_2m_max'][2], what is the 2?

Next class: Sign Language Alphabet Recogniser!